Discretization and Matrix Assembly for the Reduced PDE-Constrained OC (Ex3)

Fachseminar – Discretization Notes

January 9, 2026

Purpose

This note details the discretization process, the resulting matrices, and how they are used by the optimizers. It also states the discrete optimization problem explicitly and points to where each matrix is assembled in the provided code.

1 Continuous Problem

We consider

$$\min_{u \in L^2(\Omega)} F(u) = \tfrac{1}{2}\,\|y(u) - y_d\|^2_{L^2(\Omega)} + \tfrac{\beta}{2}\,\|u\|^2_{L^2(\Omega)}, \tag{1}$$

$$-\Delta y = \chi_\omega\, u \quad \text{in } \Omega, \qquad y = 0 \quad \text{on } \partial\Omega, \tag{2}$$

with $\Omega = (0,1)^2$, control region ω a disk centered at $(\frac{1}{2}, \frac{1}{2})$ of radius r, and $y_d(x) = 0.5 - \max\{|x - 0.5|, |y - 0.5|\}$. The indicator χ_ω satisfies $\int_\Omega \chi_\omega\, \phi\, dx = \int_\omega \phi\, dx$.

Weak form. Find $y \in H^1_0(\Omega)$ s.t. for all $v \in H^1_0(\Omega)$

$$\int_\Omega \nabla y \cdot \nabla v\, dx = \int_\Omega \chi_\omega\, u\, v\, dx. \tag{3}$$

2 Finite Element Discretization

We use conforming $P1$ elements on a triangular mesh $\mathcal{T}_h$.

- State space: $V_h \subset H^1_0(\Omega)$ with homogeneous Dirichlet boundary conditions.
- Control space: $U_h \subset L^2(\Omega)$ with the same $P1$ basis (no BCs on U_h).
- Matrices (size $n \times n$):

$$A_{ij} = \int_\Omega \nabla\varphi_i \cdot \nabla\varphi_j\, dx, \qquad \text{stiffness (Dirichlet BCs applied)}, \tag{4}$$

$$M_{ij} = \int_\Omega \varphi_i\, \varphi_j\, dx, \qquad \text{state mass}, \tag{5}$$

$$M_{U,ij} = \int_\Omega \varphi_i\, \varphi_j\, dx, \qquad \text{control-space } L^2 \text{ mass}, \tag{6}$$

$$B_{ij} = \int_\omega \varphi_i\, \varphi_j\, dx, \qquad \text{control coupling on } \omega. \tag{7}$$

The reduced mapping is $y(u) = A^{-1} B\, u$ (Dirichlet BCs applied to A only).

3 Discrete Optimal Control Problem

Let $y_d \in \mathbb{R}^n$ be the nodal vector of the desired state. The discrete objective is

$$F(u) = \tfrac{1}{2}\,(y - y_d)^T M (y - y_d) + \tfrac{\beta}{2}\,u^T M_U u, \qquad y = A^{-1} B\, u. \tag{8}$$

Equivalently in u only:

$$F(u) = \tfrac{1}{2}\,(A^{-1}Bu - y_d)^T M (A^{-1}Bu - y_d) + \tfrac{\beta}{2}\,u^T M_U u. \tag{9}$$

The reduced gradient and (Riesz) Hessian in the M_U inner product are

$$\nabla F(u) = M_U^{-1} B^T p + \beta\, u, \qquad A\,p = M\,(y - y_d), \tag{10}$$

$$Hv = M_U^{-1} B^T A^{-1} M A^{-1} B\, v + \beta\, v. \tag{11}$$

4 Where Each Matrix Is Assembled (Code)

Below we indicate the code locations that assemble the matrices.

Internal mesh version

- File: Code_v1/reduced_oc_model.py

- A: `fe.assemble(fe.dot(fe.grad(y), fe.grad(v)) * fe.dx)` (L45), then `bc.apply(A_d)` (L54).
- M_U: `fe.assemble(y * v * fe.dx)` (L46).
- B: `fe.assemble(omega * y * v * fe.dx)` with `omega` as an `Expression` for χ_ω (L51).
- Target y_d: interpolation of the given `Expression` to V (L66–68).

External mesh version

- File: Code_v2/reduced_oc_model_external.py

- B domain restriction: `B_form = uU * v * fe.dx(omega_id, subdomain_data=subdomains)` (L69).
- A: `fe.assemble(a)` with $a = \int \nabla y \cdot \nabla v\, dx$ (L73), then `bc.apply(A_d)` (L74).
- M: `fe.assemble(M)` with $M = \int yv\, dx$ (L75).
- B: `fe.assemble(B_form)` (L76).
- M_U: `fe.assemble(MU_form)` (L77).
- Mesh/tags: `load_external_mesh(...)` loads XDMF mesh and `subdomains` for `dx(omega_id)` (L20–33).

5 How Optimizers Use the Matrices

- File: Code_v1/optimizers.py (API mirrored in v2).

- **Cost**: uses fast state solves with the LU factorization of A, and quadratic terms with M/M_U.
- **Gradient**: forms p from the adjoint solve $Ap = M(y - y_d)$, then applies $M_U^{-1}B^T p + \beta u$.
- **Hessian–vector**: applies $v \mapsto M_U^{-1}B^T A^{-1} M A^{-1} B v + \beta v$.
- **Inner product**: optimizers compute norms/dots with M_U to be consistent with the Riesz map.
- **Lipschitz** L: estimated via power iteration on the Hessian operator in the M_U metric.

6 Implementation Notes

- Dirichlet BCs are enforced on A only; right-hand sides are zeroed at boundary DOFs before solves. - Factorizations of A and M_U are reused across iterations for efficiency. - In the external mesh, restricting to ω is done via `dx(omega_id, subdomain_data=subdomains)`.

7 Summary

The FE discretization yields (A, M, M_U, B) that directly define the reduced cost, gradient, and Hessian used by gradient-based optimizers. Consistency in the control-space inner product (with M_U) ensures that stepsizes (e.g., $1/L$) and convergence diagnostics are meaningful.